Assignment-1

ELL-786 MULTIMEDIA SYSTEMS

A report presented for the Assignment-1 on

Repetition code, Arithmetic code and DCT image compression

Submitted By:

Sudhanshu Chaudhary (2019JTM2207)
Santosh Kumar (2019JTM2088)

Bharti School Of

Telecommunication Technology and Management

IIT, Delhi

March 7, 2021

Contents

1 Problem Statement-1 **3**

2 Implementation **4**
2.1 Assumptions 4
2.2 Difficulties faced 4
2.3 Algorithm 4
2.4 Program Structure 5
2.5 Screenshots 6

3 Problem Statement-2 **8**

4 Implementation **9**
4.1 Algorithm 9
4.2 Program Structure 10
4.3 Difficulties faced 11
4.4 Screenshots 11

5 Appendix **15**
5.1 Appendix-A : Repetition code and Arithmetic code 15
5.2 Appendix-B : 2D DCT and 2D IDCT for image compression 22

List of Figures

1 Percentage of character changed for different d values 6
2 Comparison for no encoding , repetition code and arithmetic code 7
3 Average energy and average energy after threshold ¿350 applied 11
4 Map matrix and MSE/pixel for k=10 11
5 Original image (75.2KB) 12
6 Compression with k=23 and MSE/pixel=19.98 (33.1KB 58.6% size reduction) 12
7 Compression with k=10 and MSE/pixel=33.4 (20.8KB 72.3% size reduction) 13
8 Compression with k=6 and MSE/pixel=38.9 (15.1KB 80% size reduction) . 13
9 Compression with k=3 and MSE/pixel=44.3 (9.9KB 86.8% size reduction) . 14
10 Compression with k=1 and MSE/pixel=50.2 (5.2KB 93% size reduction) . . 14

1 Problem Statement-1

PART-1
This is a programming assignment which requires you to encode and decode binary message bits using repetition codes and arithmetic codes.

Input: A text file of size 1 KB of your choice.
Computing Environment: Matlab / Python without using built-in libraries for arithmetic code/repetition code.

Experiment 1:
1) Read the input text-file and convert it to a binary string, say of length M bits.
2) Generate a random binary error pattern of length M with hamming weight d such that the non-zero entries are uniformly distributed across M bits.
3) XOR the above error pattern with the message bits to obtain a new sequence denoted by y.
4) Using y, retrieve the text-file without any error detection/ correction coding.
5) In the decoded file, compute the percentage of modified characters with respect to the input file.
6) Repeat the above experiment by varying the value of d = 10, 100, 200, 500, 5000.

Experiment 2:
1) Read the input text-file and convert it to a binary string, say of length M bits.
2) Divide the input string into several chunks such that each chunk is of size k bits.
3) Encode each chunk into a sequence of n bits by using repetition codes and arithmetic codes. After encoding, let the total number of bits generated from the entire text-file be M.
4) Generate a random binary error pattern of length M' with hamming weight d such that the non-zero entries are uniformly distributed across M bits.
5) XOR the above error pattern with the encoded bits to obtain a new sequence denoted by y.
6) Using y, retrieve the text-file by decoding.
7) Compute the number of errors you could detect, and the number of errors you could correct.
8) In the decoded file, compute the percentage of modified characters with respect to the input file.
9) Repeat the above experiment by varying the value of d = 10, 100, 200, 500, 5000.
10) Compare the results between repetition codes, arithmetic codes and experiment-1 without any such technique.

2 Implementation

2.1 Assumptions

- Data file contain text data only.
- Repetition code use 3R and chunk size is 4.

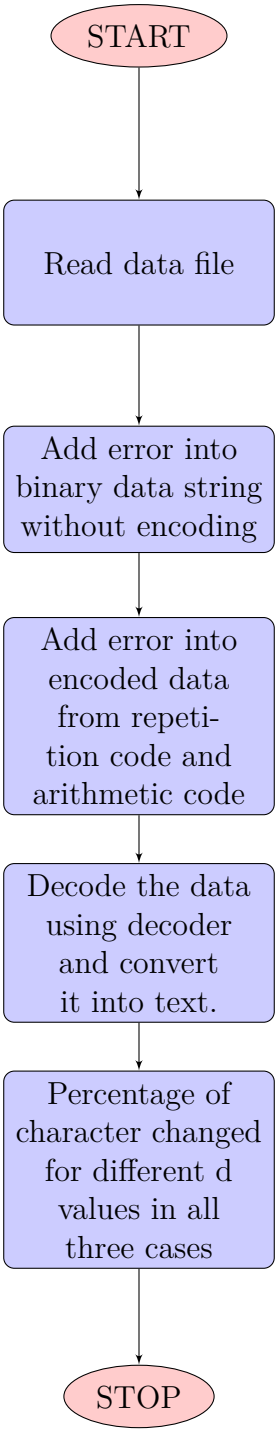
2.2 Difficulties faced

- In Arithmetic coding setting precision is difficult for long message.
- Decode the long message sometimes give different output.

2.3 Algorithm

- Firstly, import the library needed.
- Create function for uniform random error pattern with hamming weight d.
- Function to XOR two binary strings , bits to string and string to bits.
- Create function to calculate percentage of character changed.
- Create function for chunk list from string.
- Function for repetition code encoder and decoder and probability table, encoder and decoder for arithmetic coding.
- Read the file data and store it in a string .
- Sting to binary string and xor with error for different d values .
- Decode the binary string to text and calculate the character changed.
- Break binary string data to chunks and encode each chunk with repetition code encoder.
- Xor the encoded data with error decode the data using repetition code decoder.
- Decoded data to text text and calculate the character changed for different d values
- Create probability table and break text data into chunk of size 4 and encode it using arithmetic encoder.
- Convert code value to binary sting add error and break the binary string to form code value.
- Decode the text chunk of size 4 from code value using decoder function.
- Calculate the character changed for different d values

2.4 Program Structure



2.5 Screenshots

```
------------- Assignment1---------------

Experiment-1: No Encoding
Percentage of modified characters with d : 10  is  0.33233632436025257
Percentage of modified characters with d : 100  is  3.3233632436025258
Percentage of modified characters with d : 200  is  6.4473246925889
Percentage of modified characters with d : 500  is  15.021601861083417
Percentage of modified characters with d : 5000  is  80.45862412761716

Experiment-2:
Using Repetition coding
Percentage of modified characters with d : 10  is  0.16616816218012628
Percentage of modified characters with d : 100  is  1.2628780325689597
Percentage of modified characters with d : 200  is  1.694915254237288
Percentage of modified characters with d : 500  is  5.88235294117647
Percentage of modified characters with d : 5000  is  42.57228315054836

Using Arithmetic coding
Percentage of modified characters with d : 10  is  0.9970089730807578
Percentage of modified characters with d : 100  is  8.84014622798272
Percentage of modified characters with d : 200  is  16.051844466600198
Percentage of modified characters with d : 500  is  34.13094051179794
Percentage of modified characters with d : 5000  is  93.08740445330675

-------End-------
```

Figure 1: Percentage of character changed for different d values

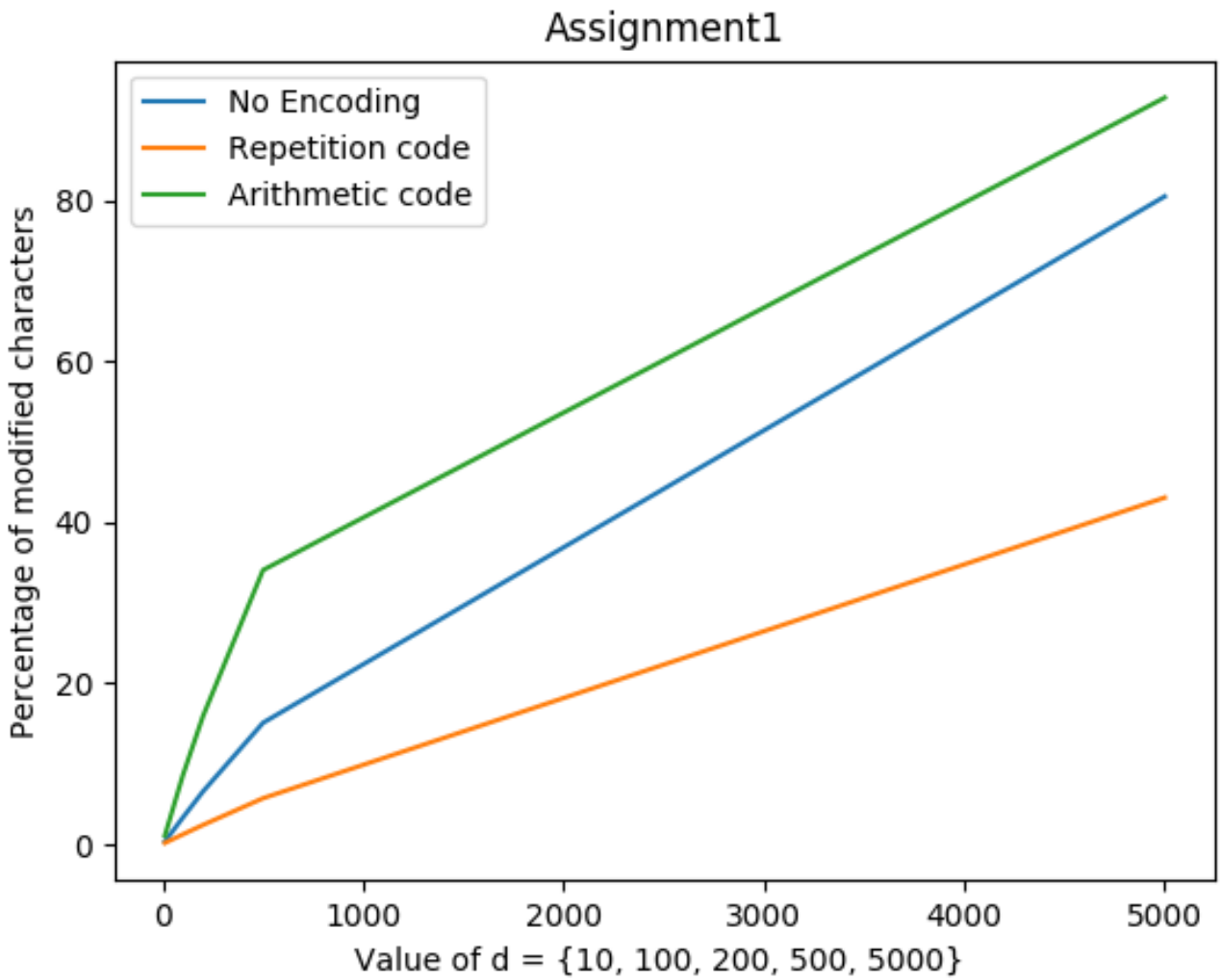


Figure 2: Comparison for no encoding , repetition code and arithmetic code

3 Problem Statement-2

PART-2
This is a programming assignment which requires you to apply Discrete Cosine Transform on an image and see how much compression you can achieve without a prominent loss.

Input:
1) A gray-scale png image (512x512 matrix)of your choice.
2) block size (typically a power of two such as 8x8 block etc...).
3) number of the coefficients to keep for each block, say 10 etc...).

Computing Environment: Matlab / Python without using built-in libraries for DCT.

Experiment 1:
1) Read the input image-file and convert it to a matrix of 8x8x say M.
2) Take the 2-D discrete cosine transform (DCT) of each 8x8 sub matrix and encode each to explain compression. You may use in-built functions like blkproc(image,[8 8],'dct2') or scipy.fftpack.dct only to verify your results, it has to be coded by you otherwise.

3) Having found the DCT coefficients of each block of the image, compute the average energy of each of the 64 coefficients across blocks. In order to do that, square the coefficient at a chosen location in each block, and sum up these values across blocks (so that energies at the same location in different blocks are summed up). Then normalize these values by the number of the blocks to find the average energy of each of the 64 coefficients. Now, we are going to "prune" some of the coefficients in each block. Hence, we need to decide which coefficients to keep. We will keep those with the highest average energy, and discard the rest (this is where the compression takes place.) You should see that most of the image's energy is concentrated around the zeroth frequency which corresponds to the upper left corner after taking the DCT transform. At first, try keeping only 10 coefficients (the 10 with the highest average energy) and see how the reconstructed image looks after taking the 2-D inverse DCT.

4) Next, vary the number of coefficients kept and see how many coefficients you can discard until the image degradation is perceptually significant. Note that the subblock size matters (try several sizes and see how the quality varies, also see below for specific block sizes and number of coefficients retained you need to try for the report).

Output parameters :
1) decompressed (the decompressed image which is the output after taking the 2D IDCT).
2) map (an index map that indicates the locations of coefficients to kept in each block. This output is a binary matrix of size "block_size x block_size", which consists of values 0 and 1 only. If map(i,j) = 1 at a certain location, this means that we are keeping all the coefficients at the (i,j)th location in each block. If map(i,j) = 0, this means that we are discarding the (i,j)th coefficient in every block) .
3) energy (a matrix which gives the average energies at each location of the block. The way

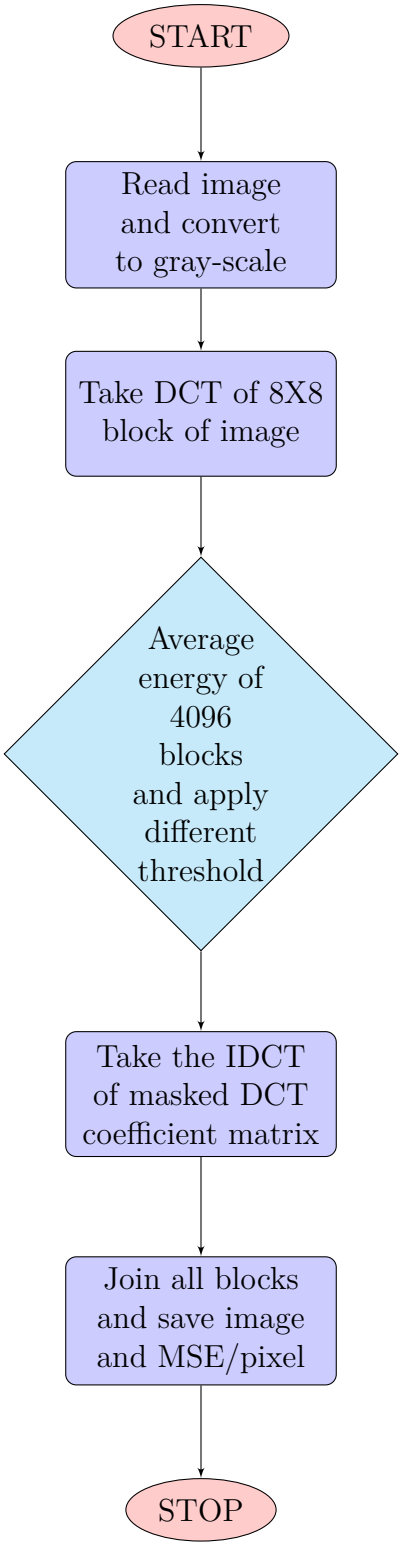
to compute the average energy for each location is explained above. This matrix should also be of size "block_size x block_size").
4) MSE/pixel.

4 Implementation

4.1 Algorithm

- Firstly, import the library needed.
- Create the transform matrix function which give N X N size matrix .
- Using the transform matrix function create the 2D DCT and 2D Inverse DCT functions.
- Create function to calculate the MSE/pixel.
- Read the image and convert it into gray scale.
- T be the transform matrix of size 8X8.
- Run the two loops over the image of dimension 512X512 with block size of 8X8.
- Take DCT of each block and square the DC coefficients and store in the energy matrix.
- Calculate the the average energy by diving the total number of blocks i.e 4096.
- Create the map matrix by applying the threshold of 350 on average energy matrix for k=10 (value less than 350 assigned 0 and greater than 350 assigned 1).
- Change the threshold to achieve high and low compression.
- Use this map matrix as mask matrix and do the element wise multiplication of DCT coefficient matrix and mask matrix.
- Take the inverse DCT of the masked DCT matrix of all the 4096 blocks.
- And create the matrix of 512 X 512 by joining all the 4096 blocks of 8X8.
- Convert the matrix into image and save it.
- Now calculate the MSE/pixel for different saved images.

4.2 Program Structure



4.3 Difficulties faced

- Using two loop for 2D DCT and 2D IDCT very slow .
- Used transform matrix multiplication method for faster conversion.

4.4 Screenshots

```
Average energy at each location of the 8 X 8 block:
[[483602   3530    874    411    192    104     66     53]
 [  2992   1254    553    260    140     80     51     40]
 [  1002    565    330    170    101     59     38     33]
 [   404    256    162    101     61     41     29     23]
 [   204    130     84     57     38     27     19     17]
 [   105     70     53     36     28     17     13     15]
 [    75     52     38     28     19     14     12     13]
 [    56     41     31     22     13     14     14     24]]

Average energy after discarding less energy coefficients:
[[483602   3530    874    411      0      0      0      0]
 [  2992   1254    553      0      0      0      0      0]
 [  1002    565      0      0      0      0      0      0]
 [   404      0      0      0      0      0      0      0]
 [     0      0      0      0      0      0      0      0]
 [     0      0      0      0      0      0      0      0]
 [     0      0      0      0      0      0      0      0]
 [     0      0      0      0      0      0      0      0]]
```

Figure 3: Average energy and average energy after threshold ¿350 applied

```
Map matrix:
[[1 1 1 1 0 0 0 0]
 [1 1 1 0 0 0 0 0]
 [1 1 0 0 0 0 0 0]
 [1 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]]


MSE/pixel:  33.38733673095703
```

Figure 4: Map matrix and MSE/pixel for k=10

Figure 5: Original image (75.2KB)

Figure 6: Compression with k=23 and MSE/pixel=19.98 (33.1KB 58.6% size reduction)

Figure 7: Compression with k=10 and MSE/pixel=33.4 (20.8KB 72.3% size reduction)

Figure 8: Compression with k=6 and MSE/pixel=38.9 (15.1KB 80% size reduction)

Figure 9: Compression with k=3 and MSE/pixel=44.3 (9.9KB 86.8% size reduction)

Figure 10: Compression with k=1 and MSE/pixel=50.2 (5.2KB 93% size reduction)

5 Appendix

5.1 Appendix-A : Repetition code and Arithmetic code

```python

import timeit
import random
import struct
import numpy as np


# Function to add error pattern uniformly with hamming weight d
def error_ran(s, w):
    n = len(s)
    s_list = []
    for i in range(n):
        s_list.append("0")
    for i in range(w):
        j = random.randint(0, n - 1)
        if s_list[j] == "0":
            s_list[j] = "1"
    list_str = "".join(map(str, s_list))
    return list_str


# Function to XOR two binary strings
def xor_string(a, b):
    m = len(a)
    n = len(b)
    result_str = ""
    if m == n:
        for i in range(m):
            if a[i] == b[i]:
                result_str += "0"
            else:
                result_str += "1"
    return result_str


# Fuction for string to 8bits binary list
def str_8bits_list(s=""):
    return [bin(ord(x))[2:].zfill(8) for x in s]


# Fuction to bits to character string
def bits2string(s):
    b = [(s[i : i + 8]) for i in range(0, len(s), 8)]
    return "".join([chr(int(x, 2)) for x in b])


# Function for percentage of character changed
def diff_letters(a, b):
    c = min(len(a), len(b))
```

```
    s = sum(a[i] != b[i] for i in range(c))
    d = len(a) - len(b)
    if d < 0:
        s = s
    else:
        s = s + d
    per_s = (float(s) / len(a)) * 100
    return per_s


# Function for creating chunks of size k
def chunk(m, k):
    chunk_list = [(m[i : i + k]) for i in range(0, len(m), k)]
    return chunk_list


# Function for Repetition Codes encoder
def repetitionCodes(data):
    res = ""
    r = 3
    for i in data:
        if i == "0":
            for j in range(r):
                res += "0"
        elif i == "1":
            for j in range(r):
                res += "1"
    return res


# Function for Repetition Codes decoder
def decodeRepetitionCodes(data):
    res = ""
    n = len(data)
    r = 3
    for i in range(0, n, r):
        if data[i] == "0":
            res += "0"
        elif data[i] == "1":
            res += "1"
    return res


def bin2float(d):
    n = len(d)
    x = 0.5
    res = 0
    for i in range(n):
        if d[i] == "1":
            res += x
        x = x / 2
    # print(res)
    return res

```

```python

def float_to_bin(num):
    return format(struct.unpack("!I", struct.pack("!f", num))[0], "032b")


def bin_to_float(binary):
    return struct.unpack("!f", struct.pack("!I", int(binary, 2)))[0]


def probrange(data):
    n = len(data)
    level = float(1) / float(n)
    str1 = list(data)
    str1.sort(reverse=True)
    d = {}
    for n in str1:
        keys = d.keys()
        if n in keys:
            d[n] = d[n] + level
        else:
            d[n] = level
    low = 0
    for k, v in d.items():
        d[k] = [low, low + v]
        low = d[k][1]
    # val1 = d[" "]
    # d[" "] = [val1[0], 1.0]
    # print(d)
    return d


def encodeAC(msg, freq):
    bincode = ""
    msglen = len(msg)
    low = 0.0
    high = 1.0
    for i in range(msglen):
        high1 = low + (high - low) * (freq[msg[i]][1])
        low1 = low + (high - low) * (freq[msg[i]][0])
        high = high1
        low = low1
        # print(low, high)
        stradle = True
        while stradle:
            # print("T")
            if low >= 0.5 and high < 1.0:
                bincode += "1"
                # print("1")
                low = 2 * (low - 0.5)
                high = 2 * (high - 0.5)
            elif low > 0 and high < 0.5:
                bincode += "0"
                # print("0")
                low = 2 * low
```

```
                high = 2 * high
            else:
                # print("stadle")
                stradle = False
    # print(bincode, low, high)
    bincode += "1"
    freqlist = []
    for k, v in freq.items():
        freqlist.append([k, v])
    # print("E", bincode)
    return bincode, (len(bincode) - 1), msglen, freqlist


def check(tag, low, high, freq):
    diff = high - low
    for i in range(len(freq)):
        per = freq[i][1][1] - freq[i][1][0]
        val = low + diff * per
        high = val
        # print("sada", low, high, tag)
        if tag < high and tag > low:
            # print("t")
            return freq[i][0], low, high
        low = val
    return freq[i][0], low, high


def decodeAC(bincode, codelen, msglen, freq):
    t = bin2float(bincode[0:codelen])
    # print(t)
    low = 0.0
    high = 1.0
    string, low, high = check(t, low, high, freq)
    # print(string, low, high)
    while len(string) != (msglen):
        if low >= 0.5 and high < 1.0:
            low = 2 * (low - 0.5)
            high = 2 * (high - 0.5)
            bincode = bincode[1:] + "0"
            t = bin2float(bincode)
            # print("111", t, low, high)
        elif low > 0 and high < 0.5:
            low = 2 * (low)
            high = 2 * (high)
            bincode = bincode[1:] + "0"
            t = bin2float(bincode)
            # print("000", t)
        else:
            # print(t, low, high)
            y, low, high = check(t, low, high, freq)
            string += y
            # print("stadle", string)
    # print(string)
    return string
```

```python


#####################################################################

print("————————— Assignment1————————————")
print()
################## Open File ########################
try:
    with open("/home/sudhanshu/Downloads/4thSem/Multimedia/As1/data.txt", "r") as fpt:
        fdata = fpt.read()
except FileNotFoundError:
    fdata = None
# print("The string data from file : ", str(fdata))

################# String to binary ##################
fdata_bn = "".join(format(ord(i), "b").zfill(8) for i in fdata)
# print(fdata_bn)


#######################################################
print("Experiment-1: No Encoding")
# # Hamming weights
list_d = [10, 100, 200, 500, 5000]
e1 = []
# Generate error pattern of length M with hamming weight d
for i in range(0, 5):
    ran_str = error_ran(fdata_bn, int(list_d[i]))
    x_str = xor_string(fdata_bn, ran_str)  # XOR  with error pattern
    rx_fdata = bits2string(x_str)  # Binary to text
    print(
        "Percentage of modified characters with d :",
        int(list_d[i]),
        " is ",
        diff_letters(fdata, rx_fdata),
    )
    e1.append(diff_letters(fdata, rx_fdata))

#######################################################
print()
print("Experiment-2: ")
print("Using Repetition coding")
e2 = []
c_data = chunk(fdata_bn, 4)  # chunk of size 4 for L2
encode_data2 = ""
for i in range(len(c_data)):
    x = str(c_data[i])
    encode_data2 += repetitionCodes(x)

# Generate error pattern of length M' with hamming weight d
for i in range(0, 5):
    ran_str1 = error_ran(encode_data2, int(list_d[i]))  # Uniform error
    x_str1 = xor_string(encode_data2, ran_str1)  # XOR
    rx_fdata1 = decodeRepetitionCodes(x_str1)  # Decode
```

```
    rx_txt1 = bits2string(rx_fdata1)
    print(
        "Percentage of modified characters with d :",
        int(list_d[i]),
        " is ",
        diff_letters(fdata, rx_txt1),
    )
    e2.append(diff_letters(fdata, rx_txt1))
#########################################################

print()
print("Using Arithmetic coding")

data = " .,zyxwvutsrqponmlkjihgfedcba"

ftable = probrange(data)
# for k, v in x.items():
#     print(k, v)

chunkdata = chunk(fdata, 4)
# print(chunkdata)

e3 = []
encodedata = []
for i in range(len(chunkdata) - 1):
    a, b, c, d = encodeAC(chunkdata[i], ftable)
    encodedata.append([a, b, c, d])

binstr = ""
for i in encodedata:
    binstr += i[0]


for i in range(0, 5):
    ran_str2 = error_ran(binstr, int(list_d[i]))  # Uniform error
    x_str2 = xor_string(binstr, ran_str2)  # XOR
    # print("val", diff_letters(binstr, x_str2))
    step = 0
    output = ""
    for j in encodedata:
        binstrchunk = x_str2[step : step + j[1] + 1]
        deco = decodeAC(binstrchunk, j[1], j[2], j[3])
        output += deco
        step = step + j[1] + 1

    print(
        "Percentage of modified characters with d :",
        int(list_d[i]),
        " is ",
        diff_letters(fdata, output),
    )
    e3.append(diff_letters(fdata, output))


```

```
#######################  Plot ##############################
# importing matplotlib module
from matplotlib import pyplot as plt


# Function to plot
plt.plot(list_d , e1, label="No Encoding")
plt.plot(list_d , e2, label="Repetition code")
plt.plot(list_d , e3, label="Arithmetic code")

plt.xlabel("Value of d = {10, 100, 200, 500, 5000} ")
plt.ylabel("Percentage of modified characters")
plt.title("Assignment1")
plt.legend()
# function to show the plot
plt.show()

print()
print("————End————")
#############################################################
```

5.2 Appendix-B : 2D DCT and 2D IDCT for image compression

```python

# Library imported
import cv2
import math
import time
import numpy as np
from scipy import fft
from PIL import Image
from datetime import timedelta

# start timer to check time
start_time = time.monotonic()

# Function to create transform matrix for DCT and IDCT
def TransformMat(m):
    T = np.zeros([m, m])
    for i in range(0, m):
        for j in range(0, m):
            cs1 = math.cos(((2 * j + 1) * i * math.pi) / (2 * m))
            if i == 0:
                T[i][j] = 1 / math.sqrt(m)
            else:
                T[i][j] = (math.sqrt(2) / math.sqrt(m)) * cs1
    return T


# Function for DCT
def dctTransform(T, TT, matrix):
    # DCT using T
    dctimag = np.matmul(np.matmul(T, matrix), TT)
    return dctimag


# Fuction for IDCT
def inverseDctTransform(T, TT, matrix):
    # inverse DCT using T
    idctimag = np.matmul(np.matmul(TT, matrix), T)
    return idctimag


# Function for MSE calculation per pixel
def MSE(mat1, mat2):
    m, n = np.shape(mat1)
    MSE = np.sum(np.square(np.subtract(mat1, mat2)))
    mseperpixel = MSE / (m * n)
    return mseperpixel


################### Main Program #######################
threshold = int(input("Enter energy threshold: "))
path = r"/home/sudhanshu/Downloads/4thSem/Multimedia/As1/cat.jpg"
```

```
pathout = "/home/sudhanshu/Downloads/4thSem/Multimedia/As1/"

img = cv2.imread(path)  # read image
wname = "imag"  # window name
cv2.imshow(wname, img)
img1 = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # convert to grayscale
# print(img1)
m, n = np.shape(img1)
T = np.array(TransformMat(8))  # tansformation matrix of size 8X8
TT = np.transpose(T)  # transform of T
energy = np.zeros([8, 8])  # create matrix to store total energy


# Run the program for with block size of 8X8 across image of 512 X 512
for i in range(0, m, 8):
    for j in range(0, n, 8):
        img2 = img1[i : i + 8, j : j + 8]
        dctimag = dctTransform(T, TT, img2)  # DCT
        energy += np.square(dctimag)  # save energy

# calculate the average energy for 4096 blocks of 8X8 size
print()
avgenergy = np.divide(energy, 4096)
print("Average energy at each location of the 8 X 8 block:")
print(np.array(avgenergy.astype(int)))
print()
# map matrix with 0 and 1 to show which
# coefficients to keep and which to  dicard
mapmat = np.zeros([8, 8])
for i in range(8):
    for j in range(8):
        if avgenergy[i][j] < threshold:  # avgenergy less than 350 discard
            avgenergy[i][j] = 0
        else:
            mapmat[i][j] = 1

print("Average energy after discarding less energy coefficients:")
print((avgenergy.astype(int)))  # avgenergy matrix after discarding
print()
# from map matrix we get masked matrix
print("Map matrix:")
Mask = mapmat.astype(int)  # map matrix
print(Mask)
print()


# Run the program for with block size of 8X8 across image of 512 X 512
for i in range(0, m, 8):
    for j in range(0, n, 8):
        img2 = img1[i : i + 8, j : j + 8]
        dctimag = dctTransform(T, TT, img2)  # DCT
        maskedimg = np.multiply(Mask, dctimag)  # mask the DCT coeff.
        idctimag = inverseDctTransform(T, TT, maskedimg)  # inverse DCT
        # join the block to form complete image
```

```
        if j == 0:
            hrow = idctimag
        else:
            hrow = np.hstack((hrow, idctimag))
    if i == 0 and j == n - 8:
        fimage = hrow
    else:
        fimage = np.vstack((fimage, hrow))

print()
finalimage = fimage.astype(np.uint8)  # final image with uint8 datatype
# print(finalimage)

img = Image.fromarray(finalimage, "L")  # create image from matrix
img.show()  # show compressed image
# img.save(pathout + "compressedcat.jpg")  # save compreesed image using T DCT
img.save(pathout + "compressedcat1.jpg")  # save compressed image

mse = MSE(img1, finalimage)
print("MSE/pixel: ", mse)
print()

end_time = time.monotonic()
print(
    "Execution time: ", timedelta(seconds=end_time - start_time)
)  # time to run the program

cv2.waitKey(0)
cv2.destroyAllWindows()
```

References

[1] *2-D Discrete Cosine Transform.* https://www.imageeprocessing.com/2013/03/2-d-discrete-cosine-transform.html.

[2] *2-D Inverse Discrete Cosine Transform.* https://www.imageeprocessing.com/2013/03/2-d-inverse-discrete-cosine-transform.html.

[3] *Arithmetic Code Discussion and Implementation.* http://michael.dipperstein.com/arithmetic/.

[4] *Lossless Data Compression Using Arithmetic Encoding.* https://neptune.ai/blog/lossless-data-compression-using-arithmetic-encoding-in-python-and-its-applications-i